Low-Level Design (LLD) Interview Cheat Sheet

Reusable reference for LLD-style pairing/technical interviews

What LLD Interviews Actually Test

Not algorithmic cleverness (that's LeetCode). LLD tests whether you can turn a fuzzy real-world system into **clean, extensible, working object-oriented code** in real time. Evaluators watch for:

- Can you scope an ambiguous problem down to something buildable in 30–40 min?
- Do you pick data structures that fit the actual access patterns?
- Are your class/interface boundaries sensible — could someone else extend this without a rewrite?
- Do you handle state, time, and edge cases correctly (not just the happy path)?
- Can you talk through tradeoffs out loud as you make decisions?

The 5-Step Approach (use every time)

1. Clarify scope (3–5 min)

- List the operations the system must support explicitly (e.g., "get, put, evict" for a cache).
- State what's explicitly out of scope (concurrency? persistence? multi-user?) — say it out loud.
- Ask about expected scale — affects data structure choice.

2. Identify core entities & data structures (5 min)

- List the nouns in the problem — these are usually your classes.
- Frequent lookups by key → hash map. Ordering + fast lookup → hash map + doubly linked list.
- Priority-based retrieval → heap. FIFO processing → queue/deque. Range queries → sorted structure.

3. Sketch class/interface design before coding (5 min)

- Write class names and public methods as stubs first.
- Decide interface vs. concrete class — "support multiple types of X" signals Strategy or Factory.
- Say out loud why you're drawing boundaries where you are.

4. Implement incrementally (20–25 min)

- Build the simplest version that satisfies core operations first — working end-to-end before edge cases.
- Narrate tradeoffs as you go.
- Don't over-engineer prematurely — add flexibility only where signaled.

5. Test & discuss extensions (5–8 min)

- Walk through 1–2 concrete examples by hand.
- Explicitly check an edge case (empty state, single element, duplicates, boundaries).
- Proactively mention extensions: thread-safety, scale beyond memory, new requirements.

Data Structure Cheat Sheet

Need	Structure	Python
O(1) key lookup	Hash map	`dict`
O(1) lookup + insertion order	Ordered hash map	`dict (3.7+) or OrderedDict`

Need	Structure	Python
O(1) lookup + O(1) add/remove both ends	Hash map + doubly linked list	`OrderedDict, or manual DLL + dict`
Priority-based retrieval (min/max)	Heap	`heapq (negate for max-heap)`
FIFO / BFS processing	Queue	`collections.deque`
LIFO / undo / matching	Stack	`list or collections.deque`
Frequency counting	Hash map	`collections.Counter`
Default values w/o key checks	Hash map	`collections.defaultdict`
Uniqueness checks	Set	`set`
Sorted iteration needed	Sorted structure	`sorted() on demand, or heapq/bisect`
Interval/range problems	Sorted list + merge logic	`bisect module`

Common LLD Problems & Their Core Pattern

- **LRU Cache** → OrderedDict (or hash map + manual doubly linked list) + capacity eviction
- **LFU Cache** → hash map key→value, hash map key→freq, hash map freq→ordered keys
- **Rate Limiter** → token bucket (counter + timestamp) or sliding window (deque of timestamps)
- **Parking Lot** → classes per spot type, manager class matching vehicles to spots, Strategy pattern for pricing
- **Elevator System** → state machine per elevator (idle/moving/door open), dispatcher chooses which elevator serves a request
- **Booking/Scheduling System** → interval structure to detect overlaps, state machine for booking status
- **Notification/Dispatch System** → Observer pattern (subscribers per event type), queue for async-style dispatch
- **In-memory KV Store with TTL** → hash map + min-heap (or lazy expiration check) keyed by expiration time
- **Pub-Sub System** → hash map of topic → list of subscriber callbacks, Observer pattern
- **Tic-Tac-Toe / Board Games** → grid representation + win-condition checker, decoupled from game loop
- **Vending Machine** → state machine (idle/selecting/dispensing/returning change)

Design Patterns Worth Naming

- **Strategy** — interchangeable algorithms behind a common interface
- **Observer** — subscribers notified of state changes
- **Factory** — centralize creation of related object types
- **Singleton** — single shared instance (mention, don't force)
- **State** — explicit state machine w/ defined transitions
- **Decorator** — wrap behavior w/o modifying original class

Name the pattern out loud when you use it — signals fluency without over-architecting.

Python-Specific Class Tips

- `@dataclass` for simple value classes — auto __init__/__repr__/__eq__
- `abc.ABC` + `@abstractmethod`, or `typing.Protocol`, for multi-type interfaces
- Implement `__lt__`/`__eq__` for classes going into a heapq
- Use `Enum` for fixed states instead of raw strings
- Type hints on signatures cost nothing, signal clean interface thinking

---

Things to Say Out Loud

- “Let me clarify the scope before I start coding...”
- “I'm choosing X here because the access pattern is mostly Y...”
- “I'll start with the simplest version, then layer in edge cases.”
- “This is essentially the [pattern name] pattern.”
- “If this needed to support concurrency/scale, here's what I'd change...”
- “Let me test this with a quick example before moving on.”

Edge Cases to Always Consider

- Empty state (empty cache, no bookings, zero elements)
- Single element
- Duplicate keys/values
- Capacity boundaries (cache at max size, rate limit at threshold)
- Operations on non-existent keys/entities
- Time-based edge cases (expiration at boundary, simultaneous timestamps)